OAU EEE: API and ESP32 coding cheat sheet

Owner: API, firmware and backend contributors. Working proposal, 2 October 2026.

This guide explains where the API runs, what the ESP32 runs and how both sides use one contract.

Where the API lives

The API runs on the Django server. It exposes HTTPS endpoints that accept and return agreed JSON messages.

The ESP32 runs firmware containing Wi-Fi/network code, a sensor driver and an API client. Loading firmware onto the device does not move the server or database onto it.

The website communicates with Django through staff pages or selected API routes. Django accesses PostgreSQL. The ESP32 never receives database credentials.

This is the proposed project arrangement. Actual sensor, network and authentication mechanisms need team decisions.

Responsibilities by location

Location	Code responsibility
Browser	Staff login, request a check, display permitted outcome
Django API	Authenticate sender, validate JSON, coordinate services
Identity services	Request freshness, retries and credential resolution
PostgreSQL	Devices, mappings, requests and minimal event records
ESP32 firmware	Obtain request, capture/match through biometric subsystem, post outcome
Biometric subsystem	Template storage and matching, under separate reviewed design

A fingerprint outcome identifies a student only after mapping checks. Staff permission still controls academic access.

Proposed first integration flow

1. Authorised staff creates an identity check for a selected device.
2. The server creates `request_id`, an unpredictable challenge and `expires_at`.
3. The device polls for its assigned request over authenticated HTTPS.
4. The biometric subsystem reports matched, `no_match` or error.
5. Firmware posts an outcome with the request and event identifiers.
6. The server validates freshness and resolves its active credential mapping.
7. Staff retrieves the result and then separately requests permitted student records.

Polling is a simple initial proposal. Push/MQTT needs a separately agreed contract and operational design.

Retained and newly proposed names

Retain handbook message keys: `contract_version`, `device_id`, `request_id`, `event_id`, challenge, outcome, `credential_ref` and `diagnostic_code`.

Retain `student_id` as the stable Student UUID. Do not send a freely chosen `student_id` as match evidence.

Proposed additions: `expires_at`, `occurred_at`, `received_at`, `processing_status`, `verification_status` and `correlation_id`.

Use outcome for the sensor report and `verification_status` for the server's conclusion. A reported matched outcome may still be unresolved or rejected.

Contract field types

Field	Wire type	Rule
`contract_version`	string	Exactly identity-v1
`device_id`	string, max 100	Registered ID bound to authenticated principal
`request_id`, `event_id`	UUID string	Server request key, client-generated event key
challenge	opaque string	Server unpredictable value, compared to stored digest
outcome	string enum	matched, `no_match`, error
`credential_ref`	string, max 100	Required only for matched, device-scoped
`diagnostic_code`	string enum	Required for error, controlled code
`occurred_at`	ISO 8601 string	Optional device claim, never sole freshness proof

Contract field types (continued)

Demo IDs in the handbook are readable placeholders. UUID wire IDs here are a proposed concrete replacement.

Identity database dictionary

Proposed Device field	Django/PostgreSQL type	Meaning
`device_id`	`CharField(100)` PK / varchar	Registered external device label
`display_name`	`CharField(100)`	Staff-visible label
`is_active`	`BooleanField` / boolean	Operational availability
`revoked_at`	`DateTimeField` nullable / timestamptz	Revocation evidence
`credential_version`	`PositiveIntegerField` / integer	Rotation version, if selected auth uses it
`last_seen_at`, `created_at`	`DateTimeField` / timestamptz	Nullable last contact, server creation time

Identity database dictionary (continued)

The credential secret belongs in approved protected provisioning/storage. Do not store plaintext secrets in fixtures or log them.

Credential mapping dictionary

`CredentialMapping` fields: `mapping_id` UUID PK, device FK Device, `credential_ref` `CharField(100)`, student FK Student, `is_active` `BooleanField`, `enrolled_at` `DateTimeField`, `revoked_at` nullable `DateTimeField`.

`credential_ref` identifies a credential in the agreed device/biometric namespace. It is not a fingerprint template or matriculation number.

Candidate uniqueness: one active mapping for (device, `credential_ref`). Historical retired mappings remain distinguishable.

If devices share a sensor database, model that shared scope explicitly. Initial device scope is a proposal, not a claim about the chosen hardware.

Request and event dictionary

`IdentityCheckRequest`: `request_id` UUID PK, device FK, `requested_by` FK staff, `challenge_digest` `CharField(64)`, `created_at/expires_at` `DateTimeField`, `consumed_at` nullable `DateTimeField`, status `CharField(32)`.

`IdentityEvent`: `event_id` UUID PK, request FK, device FK, outcome `CharField(16)`, `credential_ref` optional `CharField(100)`, `verification_status` `CharField(32)`, student FK nullable, `diagnostic_code` `CharField(80)`, `received_at` `DateTimeField`.

Add a canonical payload digest if needed for repeat comparison. Avoid persisting the raw challenge. Proposed successful status: verified. Other terminal statuses: `no_match`, unresolved, error.

Proposed endpoint map

Caller	Method and route	Purpose
Staff	POST `/api/identity/v1/checks/`	Create authorised request
Device	GET `/api/identity/v1/devices/me/checks/next/`	Obtain its pending request
Device	POST `/api/identity/v1/outcomes/`	Submit one event
Staff	GET `/api/identity/v1/checks/{request_id}/`	Read permitted check state

These endpoints do not exist yet. Routes, response codes and polling interval need contract review.

Proposed endpoint map (continued)

Ordinary staff pages can use Django views. A separate single-page website is not required to use the identity API.

Staff request and device challenge

Staff request JSON: {"device_id": "D-DEMO-1"}. The server derives requested_by from the authenticated staff context.

Server returns 201 with request_id and expires_at. Device polling returns 200 with contract_version, device_id, request_id, challenge and expires_at, or 204 when none is pending.

Create challenge with a cryptographic random generator such as secrets.token_urlsafe. The server owns expiry and device assignment.

Proposed target_student_id, if a one-to-one verification workflow needs it, is an optional staff-only claim. A mismatch must fail verification, never rewrite the mapping.

Matched outcome example

```
{
"contract_version": "identity-v1",
"device_id": "D-DEMO-1",
"request_id": "11111111-1111-4111-8111-111111111111",
"event_id": "22222222-2222-4222-8222-222222222222",
"challenge": "fictional-challenge-do-not-use-live",
"outcome": "matched",
"credential_ref": "C-DEMO-7"
}
```

All values are fictional. Device authentication belongs in the selected authentication mechanism, separate from body claims.

No-match and sensor-error shapes

`no_match` uses the same `contract_version`, `device_id`, `request_id`, `event_id` and challenge, with outcome="`no_match`" and no `credential_ref`.

error uses outcome="error", `diagnostic_code="sensor_unavailable"` and no `credential_ref`. Additional allowed codes may include `capture_timeout` and `sensor_failure`.

The serializer rejects blank matched credentials, unexpected `credential_ref` on `no_match`/error and unknown diagnostic codes. Never interpret sensor errors as a match.

Strictly reject or explicitly define handling of unknown keys. Both simulators must apply the same rule.

Minimal replies and staff results

Device acknowledgement proposal: {"event_id": "22222222-2222-4222-8222-222222222222", "processing_status": "accepted"}.

accepted means the event was processed. It does not promise verified identity. The device does not need academic records, guardian contacts, scores or CGPA.

Staff check response proposal: request_id, status, verification_status, student_id only when verified, diagnostic_code when relevant.

Staff access to that student record still calls permission-aware students/academics queries. UUID knowledge and successful fingerprint matching do not grant permission.

Authentication design choice

Recommended initial contract assumption: HTTPS with validated server certificate and a unique, revocable credential per device. Staff uses Django session authentication and CSRF for modifying requests.

The team must select device credential transport and storage. Candidate mechanisms include a per-device token or mutual TLS, depending on provisioning and hardware constraints.

Custom `DeviceAuthentication` resolves the registered principal. It verifies that JSON `device_id` equals that principal and checks active/revoked state.

A fixed shared password across devices defeats individual revocation. A body `device_id` alone cannot authenticate a sender.

Freshness, retries and replay

Bind `request_id`, challenge and device together. Use server time for `expires_at`. Check device/mapping state again at processing.

The same `event_id` with the same accepted payload returns the stored acknowledgement. The same `event_id` with different content returns conflict.

A new `event_id` cannot reuse a consumed challenge to create a second result. Lock the request while consuming it and commit event plus required audit atomically.

If an acknowledgement is lost, firmware retries the same `event_id` and payload. A device timestamp or new random ID does not make old evidence fresh.

Proposed HTTP failure contract

HTTP	Stable diagnostic	Caller action
400	`invalid_payload`, `unsupported_contract`	Correct payload/version, no blind retries
401	`authentication_required`, `invalid_device_credential`	Reprovision or correct authentication
403	`device_revoked`, forbidden	Stop operation, request authorised review
409	`event_payload_conflict`, `request_consumed`	Preserve event, resolve conflict
410	`request_expired`	Obtain a new authorised request
429	`rate_limited`	Respect Retry-After

Proposed HTTP failure contract (continued)

HTTP	Stable diagnostic	Caller action
503	`identity_unavailable`	Bounded retry of the same event

Unknown/revoked mapping yields an accepted unresolved business outcome with no student ID. Database failure yields unavailable, never verified.

app/identity/ code map

models.py: Device, CredentialMapping, IdentityCheckRequest and IdentityEvent.

services/requests.py: create_check_request, get_pending_request, validate_request_freshness. services/outcomes.py: process_outcome and idempotency checks.

services/resolution.py: resolve_credential(*, device_id, credential_ref) -> ResolutionResult. queries.py: active scoped mappings and permitted staff check reads.

api/authentication.py: selected DeviceAuthentication. api/permissions.py: staff/device permission classes. api/serializers.py: CheckRequestSerializer, OutcomeSerializer and reply serializers. api/views.py and urls.py coordinate routes.

Server function contracts

`create_check_request(*, actor, device_id) -> IdentityCheckRequest`. Requires permitted staff, active device and server-generated challenge.

`process_outcome(*, authenticated_device, payload) -> OutcomeAck`. Requires validated shape, bound request, freshness, idempotency, mapping resolution and audit.

`resolve_credential(*, device_id, credential_ref) -> ResolutionResult`. Distinguishes verified, `unknown_mapping`, `mapping_revoked` and unavailable.

`get_check_for_view(*, actor, request_id) -> permitted` response. No mutable writes hidden inside read helpers. Inputs cannot supply an authoritative actor or an arbitrary verified student ID.

ESP32 firmware code map

Proposed components: `network_client` manages Wi-Fi and HTTPS, `identity_api_client` encodes requests/replies, `sensor_adapter` talks to the biometric sensor, `identity_workflow` coordinates capture and delivery.

config supplies server URL and reviewed timeout values. provisioning owns individual device `identity/credentials.` outbox retains pending `event_id` and payload until acknowledgement or terminal failure.

Use the chosen ESP-IDF/Arduino project's existing file conventions once selected. These component names do not prescribe a specific build system.

Checkpoint: verify server certificate, bounded memory/payloads, sensor errors, reconnect, lost acknowledgement, device restart, credential rotation and no secret logging.

OOP and interfaces for integration

Define `SensorAdapter` methods for `capture_and_match` and controlled error output. A fake adapter lets firmware/workflow tests run without the physical sensor.

Define `IdentityApiClient` methods `fetch_pending_check` and `submit_outcome`. Replace transport with a fake in tests without changing workflow rules.

The server uses model classes for storage and small result dataclasses for service outputs. Keep protocol validation separate from credential resolution.

Use functions for stateless JSON conversion and checks. Interfaces help change sensor or transport without renaming shared contract fields.

Contracts and simulator files

`contracts/identity-v1/README.md`: endpoints, authentication, field meanings, limits and compatibility rules. outcome.schema.json: strict executable shape if JSON Schema is selected.

examples/: matched.json, no-match.json, error.json, invalid-missing-credential.json and unknown-version.json. errors.md and CHANGELOG.md explain outcomes and changes.

`simulators/terminal/`: sends fixtures to real software. `simulators/receiver/`: checks firmware messages and returns fixture replies. `fixtures/synthetic/`: fake devices and mappings.

Never enable simulator trust shortcuts in production. Run the same fixtures against both implementations.

API completion checkpoints

`test_device_auth.py`: missing/invalid credentials, device-body mismatch, revocation and forbidden staff.

`test_freshness.py`: expired, wrong-device and consumed challenge. `test_resolution.py`: active/unknown/revoked mapping and identical local refs on two devices.

`test_minimal_reply.py`: no unnecessary profile/academic fields. `test_unavailable.py`: database outage never returns verified identity.

Also test identical retry, altered-event conflict, concurrent submissions, schema/version rejection and staff result scope. Physical matching accuracy requires separate biometric tests.

Configuration and naming conventions

Proposed constants: `IDENTITY_REQUEST_TTL_SECONDS`, `DEVICE_POLL_INTERVAL_SECONDS`, `MAX_OUTCOME_BYTES` and `IDENTITY_API_VERSION`. Choose values from measured requirements.

Proposed environment keys: `IDENTITY_API_BASE_URL` and `DEVICE_ID` for firmware deployment configuration. Keep device credentials in the selected provisioning mechanism.

Use `snake_case` JSON keys, descriptive verbs for functions and explicit units for durations. Python None, JSON null and SQL NULL describe absence. UUID and timestamp objects cross JSON as strings.

Version breaking field/authentication changes explicitly. Deploy server and firmware compatibility together, using contract fixtures as evidence.

First integration demonstration

Create a fake active device, one fake student and one device-scoped credential mapping. Staff requests a check. The terminal simulator sends matched and staff obtains the permitted student reference.

Repeat the identical event safely. Show `no_match`, expired challenge, revoked device, unknown mapping and unavailable database behaviour.

Firmware can then use the receiver simulator before connecting to Django. Connect the real device only after certificate/authentication and restart/retry checks pass.

This milestone proves communication and resolution. It does not establish biometric accuracy or departmental production approval.

Open decisions and references

Agree sensor/template location, matching scope, one-to-many identification versus claimed-student verification, device authentication, provisioning, clock/expiry, polling, payload limits and offline behaviour.

Project basis: supplied walkthrough, handbook Section 13 and current README. Endpoints, new entity/field names and authentication choice here are proposals.

ESP HTTPS client mechanics: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/esp_http_client.html

API authentication mechanics: https://www.django-rest-framework.org/api-guide/authentication/

Choose documentation versions matching pinned dependencies. Departmental and biometric decisions remain with the appropriate reviewers.